FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Leonardo Ricci Santos Oliveira

Produto: **ProvaPronta**
https://github.com/leonardoricci-tsi/ProvaPronta

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 10/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 5
3.5 Modelo de dados do domínio 6
3.6 Segurança 7
3.7 Observabilidade, CI/CD e documentação da API 8
3.8 Camadas IaaS, PaaS e SaaS do projeto 8
4 Quadro de entregas 8
5 Atividades realizadas 9
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 9
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 12
5.3 Atividade 9 — Fork e plano de negócio 15
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 19
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 22
6 Considerações finais 25
7 Declaração de autoria 26

1 Introdução

A disciplina de Introdução à Computação em Nuvem foi construída em cima de um único projeto que cresceu a cada atividade. No primeiro bimestre ele era um catálogo de filmes do Tom Hanks: uma API em FastAPI que foi ganhando login separado num microsserviço, controle de acesso por papel, auditoria com Redis, upload de arquivos num object storage, CI/CD e observabilidade, tudo rodando numa stack do Portainer na VM da disciplina, com um subdomínio próprio. No segundo bimestre a ideia mudou de escala: em vez de só aprender as peças, transformar o projeto num produto de verdade, com plano pago, documentação, plano de negócio e assinatura recorrente.

O produto que escolhi é o **ProvaPronta**, um SaaS para professores. A dor vem de perto: um professor da rede pública com várias turmas passa horas montando prova no Word, imprimindo, corrigindo à mão e lançando nota em planilha, quase tudo fora do horário pago. No ProvaPronta o professor organiza turmas e alunos, guarda os PDFs do material numa biblioteca, gera questões de múltipla escolha com IA a partir desses PDFs, cria uma versão B embaralhada contra cola e imprime uma folha de respostas por aluno, com QR code, pensada para ser corrigida pela foto do celular. A cobrança é feita pelo Stripe, com um plano Free, o Professor Pro e o plano Escola para a coordenação.

Este relatório tem duas partes. A primeira descreve a arquitetura do sistema que está no ar hoje em leonardo-oliveira-isw055.lapps.studio: componentes, onde rodam, o caminho de uma requisição, o fluxo da assinatura, o modelo de dados, a segurança, a observabilidade e as camadas de nuvem. A segunda presta contas das atividades 7 a 11, com a data planejada, a data em que cada uma foi entregue e a evidência de cada entrega.

2 Metodologia

As atividades 7 e 8 foram feitas no repositório público do catálogo de filmes (`api_filme_`), o mesmo do 1º bimestre. Na atividade 9, esse repositório foi copiado com todo o histórico (`git clone --bare` + `git push --mirror`) para um repositório novo, com o nome do produto, onde o SaaS foi desenvolvido nas atividades 10 e 11, na mesma stack do Portainer e no mesmo domínio. A data e a hora de cada entrega foram extraídas do histórico de commits (`git log`), conferidas na página do commit no GitHub e, da atividade 9 em diante, na issue de entrega que menciona o professor. Os prints mostram o README com a menção ao professor, o histórico de commits e o sistema em execução.

O desenvolvimento foi feito localmente no meu Mac, com o Docker Compose subindo a mesma stack de produção (api, auth-service, log-service, Redis e Garage) e um MySQL local; os testes automatizados rodam com pytest num container Python, e o frontend em Angular é compilado para dentro da imagem da api. Usei o Claude Code como assistente de programação ao longo do bimestre, revisando e testando o que era gerado antes de cada commit. Cada entrega seguiu o mesmo caminho: commit no `main`, GitHub Actions rodando os testes, o build das imagens e um smoke test com a stack inteira, publicação das imagens no GHCR e o Portainer puxando a versão nova por GitOps. Quando a entrega mudava o banco, a migration do Alembic era aplicada pelo console do container da api no Portainer. As partes externas (Stripe em modo de teste, domínio de e-mail no Resend, variáveis da stack) foram configuradas nos painéis de cada serviço.

Como o GitHub só mostra a hora exata de um commit ou de uma issue ao passar o mouse sobre a data, a evidência de cada atividade junta dois prints: a página do commit ou da issue de entrega (repositório, hash ou número, título) e o mesmo registro aberto pela API pública do GitHub (`api.github.com`), que mostra a data e a hora exatas. A API usa o horário UTC; no quadro de entregas as horas estão convertidas para o horário de Brasília (UTC−3).

3 Arquitetura do SaaS

3.1 Visão geral e componentes

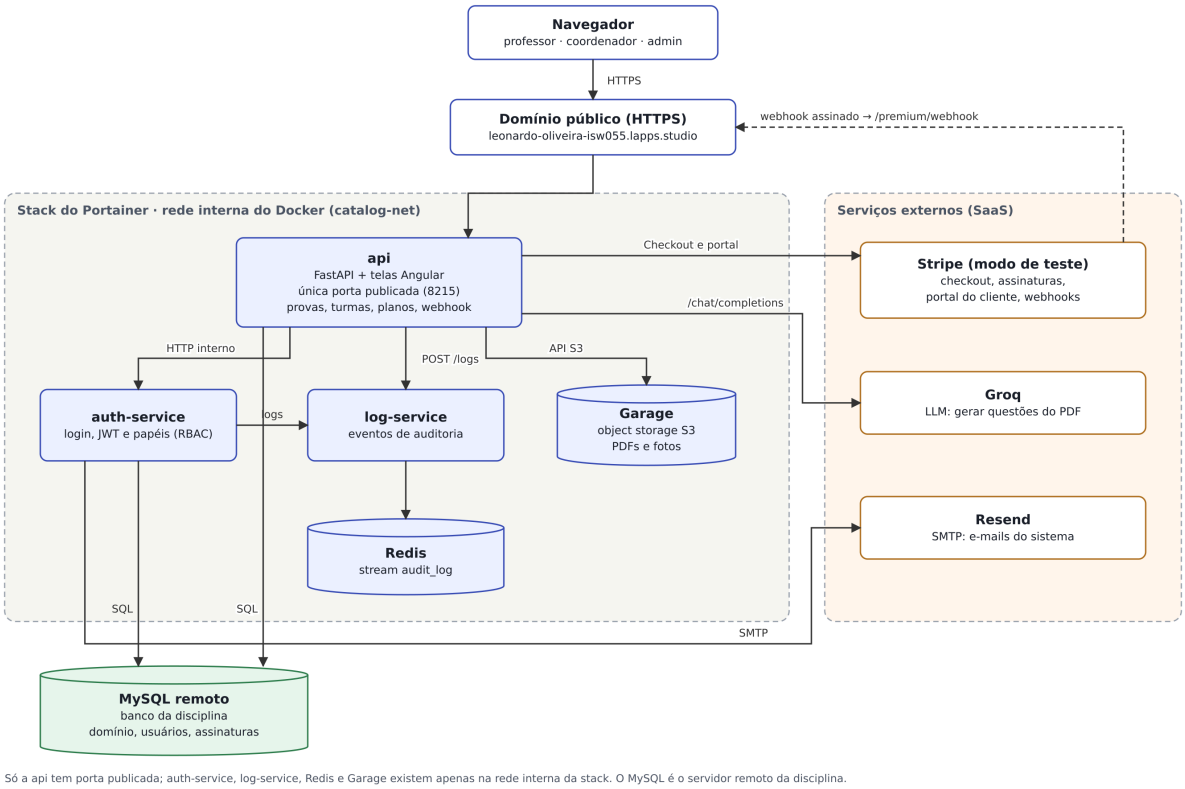


Figura 1: Componentes do ProvaPronta: o que é público, o que fica só na rede interna da stack e os serviços externos

Componente	Responsabilidade	Tecnologia / imagem	Público?
api	telas (Angular), API do negócio, landing, webhook do Stripe, leitura da folha por foto	FastAPI · `provapronta`	sim (domínio, porta 8215)
auth-service	cadastro, login, JWT, papéis (RBAC), e-mails	FastAPI · `provapronta-auth`	não (só rede interna)
log-service	eventos de negócio e auditoria	FastAPI · `provapronta-log`	não
Redis	stream `audit_log` com os eventos	`redis:7-alpine`	não
MySQL (no lugar do MariaDB)	domínio, usuários, assinaturas	servidor remoto da disciplina	não (fora da stack)
Garage (no lugar do MinIO)	PDFs das apostilas e fotos de perfil, bucket privado	`provapronta-garage` (S3)	não; link pré-assinado via api

Componente	Responsabilidade	Tecnologia / imagem	Público?
Stripe (externo)	checkout, assinaturas, portal, webhooks	serviço SaaS, modo de teste	webhook → api
Groq e Resend (externos)	LLM para gerar questões; SMTP dos e-mails	serviços SaaS	não

3.2 Onde roda

O sistema roda na mesma stack do Portainer usada desde o 1º bimestre, `api-tom-hanks-leonardo`, na VM da disciplina e no mesmo subdomínio. Na atividade 10 a stack passou a puxar o código do repositório novo, `leonardoricci-tsi/ProvaPronta`; como o Portainer CE não deixa trocar a URL do Git de uma stack, ela foi removida e recriada com o mesmo nome, as mesmas variáveis e os mesmos volumes. O arquivo usado é o `docker-compose.portainer.yml`, e o Portainer verifica o repositório a cada 5 minutos (GitOps).

São cinco containers: `api` (imagem `ghcr.io/leonardoricci-tsi/provapronta`), `auth-service` (`provapronta-auth`), `log-service` (`provapronta-log`), `redis` (`redis:7-alpine`) e `garage` (`provapronta-garage`). Todos têm health check. Só a api publica porta (8215, para onde o proxy HTTPS do domínio aponta); os outros quatro existem apenas na rede interna `catalog-net`. O compose não define limite de memória por container: os containers usam a memória disponível da VM. Os dados ficam em volumes nomeados (`redis-data`, `garage-meta`, `garage-data`) e no MySQL remoto da disciplina, que fica fora da stack.

As variáveis de ambiente da stack são: `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRE_MINUTES`, `AUTH_SERVICE_URL`, `LOG_SERVICE_URL`, `REDIS_URL`, `CATALOG_PUBLIC_URL`, `S3_ENDPOINT_URL`, `S3_PUBLIC_URL`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`, `S3_BUCKET`, `GARAGE_RPC_SECRET`, `GARAGE_DEFAULT_ACCESS_KEY`, `GARAGE_DEFAULT_SECRET_KEY`, `GARAGE_DEFAULT_BUCKET`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM_EMAIL`, `SMTP_FROM_NAME`, `STRIPE_SECRET_KEY`, `STRIPE_PRICE_PRO`, `STRIPE_PRICE_ESCOLA`, `STRIPE_WEBHOOK_SECRET`, `LLM_API_KEY`, `LLM_BASE_URL` e `LLM_MODEL`. Os valores ficam só no Portainer. A URL pública é https://leonardo-oliveira-isw055.lapps.studio, com a documentação da API em `/docs`.

3.3 Fluxo de uma requisição

A ação mais comum do produto é o professor criar uma prova. O caminho completo é:

1. O navegador, já logado, envia `POST /provas` para o domínio, com o JSON da prova e o cabeçalho `Authorization: Bearer <JWT>`. O token foi emitido pelo auth-service no login e fica guardado no navegador.
2. O proxy HTTPS da disciplina recebe a requisição no subdomínio e repassa para a porta 8215 da api.
3. A api valida o JWT localmente, com o mesmo `JWT_SECRET` do auth-service, e lê dele o id do usuário e o papel (`professor`, `professor_pro`, `coordenador` ou `admin`). Token ausente ou inválido responde `401`. O auth-service não precisa ser chamado a cada requisição; ele é consultado para login, cadastro, troca de papel e dados de usuário.
4. A api confere no MySQL se a turma informada é do próprio usuário, com o filtro `usuario_id` na consulta. Se for de outra pessoa, responde `404`, como se não existisse.
5. Se o papel é Free, a api conta as provas criadas no mês. Na terceira responde `403` com a mensagem do limite e registra `acesso_negado:limite_free:provas`.
6. A api grava a prova e as questões no MySQL numa transação.

7. A api manda o evento `prova_criada:{id}` para o log-service (`POST /logs`), sem esperar resposta: se o log cair, a prova é criada do mesmo jeito. O log-service grava o evento no stream `audit_log` do Redis.
8. O navegador recebe `201 Created` com a prova em JSON.

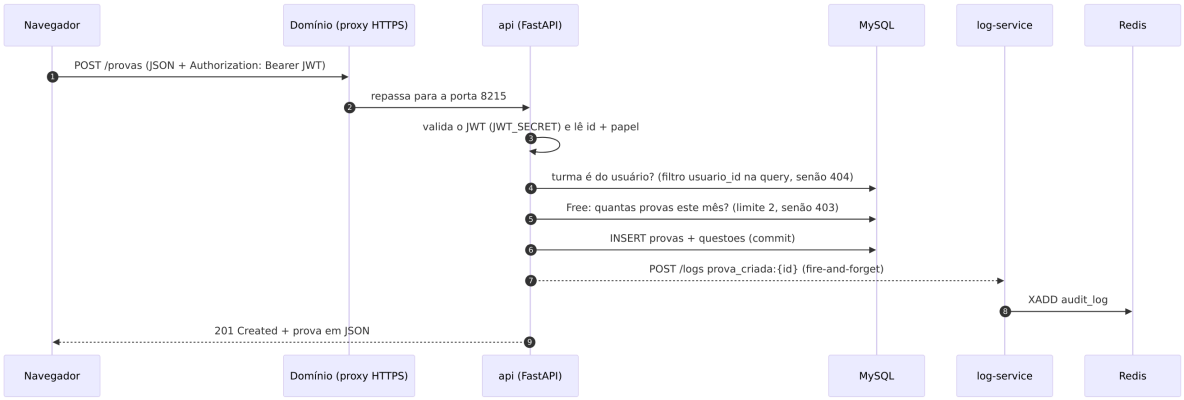


Figura 2: Fluxo de uma requisição: o professor cria uma prova (POST /provas)

3.4 Fluxo de assinatura

Os planos são o Free (padrão de todo cadastro, sem passar pelo Stripe), o Professor Pro (R$ 29,90 por mês) e o Escola (R$ 199,90 por mês), os dois pagos criados no Stripe como produtos com preço recorrente mensal. Cada plano é um papel do RBAC: assinar o Pro troca o papel de `professor` para `professor_pro`.

Quando o professor clica em "Assinar", a api cria, uma única vez, o customer do Stripe dele e guarda o ID na tabela `assinaturas`; depois abre uma Checkout Session com `mode=subscription` e devolve a URL. O cartão é digitado na página do Stripe e nunca passa pelo servidor. A volta do navegador (`?checkout=sucesso`) não ativa nada: quem muda o plano é o webhook, e a tela fica consultando `GET /premium/status` até ele chegar.

O webhook chega em `POST /premium/webhook`. A primeira coisa é validar o cabeçalho `Stripe-Signature`, um HMAC do corpo bruto com o segredo do endpoint e um timestamp: sem assinatura válida a resposta é `400` e nada é gravado. Depois a api trata quatro eventos:

- `checkout.session.completed`: troca o papel no auth-service e grava na tabela `assinaturas` o customer, a assinatura, o status `active` e o valor mensal;
- `customer.subscription.updated`: atualiza plano, status e valor. O plano é lido pelo preço, então uma troca de Pro para Escola feita no portal também vale. Status fora de `active` e `trialing` (por exemplo `past_due`) volta a conta para o Free;
- `customer.subscription.deleted`: a assinatura acabou, a conta volta para o Free;
- `invoice.payment_failed`: marca `past_due`, volta para o Free na hora, manda um e-mail de aviso e mostra um alerta em todas as telas. Se o pagamento passar depois, o `updated` com `active` devolve o plano.

Se o mesmo evento chegar duas vezes, a segunda entrega é ignorada: o ID de cada evento processado fica na tabela `eventos_stripe`. Se um evento falhar no meio (por exemplo, o auth-service fora do ar), a api responde `502`, o evento não é marcado e o Stripe reenvia. Uma assinatura cancelada também não é reativada por um `updated` que chegue atrasado. O cancelamento é feito pelo botão "Gerenciar assinatura", que abre o portal do cliente do Stripe, configurado para cancelamento imediato. Com o plano novo no papel, a próxima requisição da api já não aplica o limite do Free.

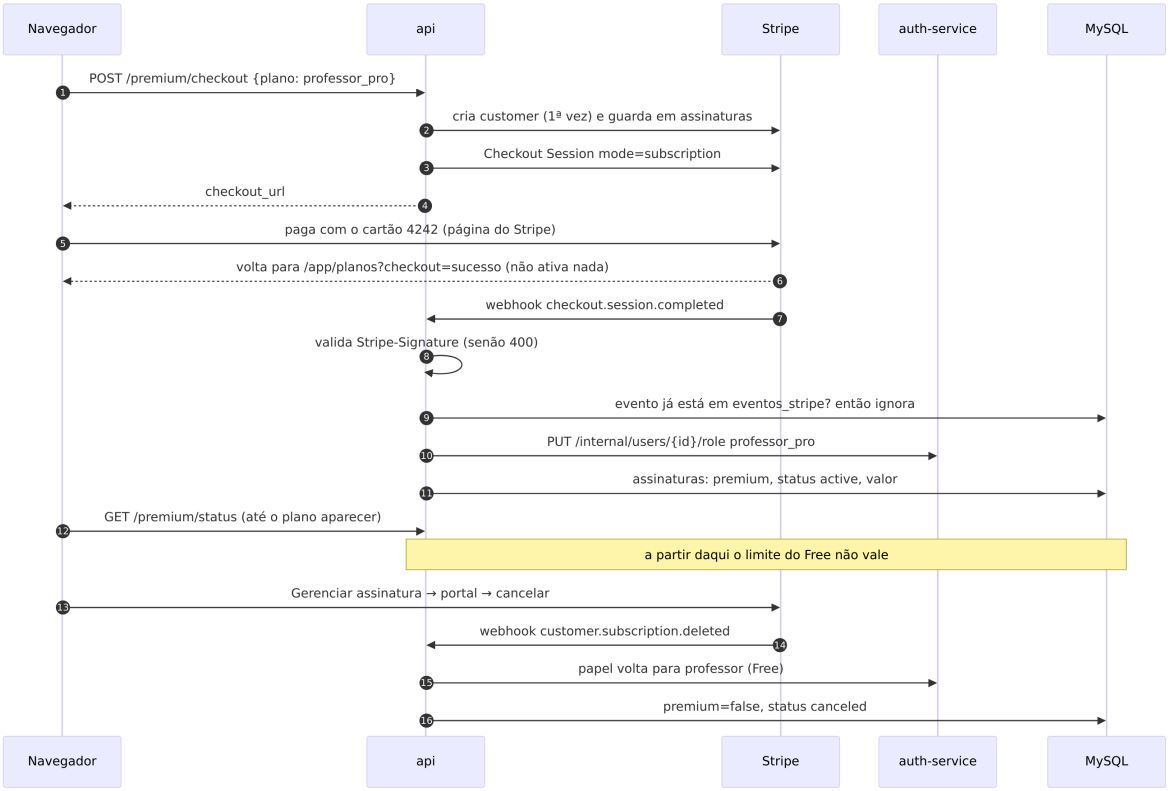


Figura 3: Fluxo de assinatura: checkout ⟶ webhook validado ⟶ plano no banco ⟶ cancelamento pelo portal

3.5 Modelo de dados do domínio

O domínio tem quatro tabelas principais com chave estrangeira: `turmas`, `alunos` (FK `turma_id`, apagada junto com a turma), `provas` (FK `turma_id` com `RESTRICT`: não dá para apagar uma turma que tem prova) e `questoes` (FK `prova_id`, apagada junto com a prova). A biblioteca de materiais tem `pastas` e `materiais` (FK `pasta_id`). O PDF e as fotos ficam no Garage; no banco fica só a chave do objeto.

A tabela `usuarios` pertence ao auth-service. As tabelas da api guardam o `usuario_id` sem chave estrangeira para ela, porque cada serviço é dono das próprias tabelas; a ligação é lógica e é garantida pelo JWT. É o `usuario_id` que liga o usuário ao plano: o papel (`role`) fica em `usuarios`, e a tabela `assinaturas` (chave `usuario_id`) guarda `stripe_customer_id`, `stripe_subscription_id`, `plano`, `status` e `valor_mensal_centavos`. A tabela `eventos_stripe` guarda os IDs dos eventos já processados. O esquema é versionado por migrations do Alembic (`alembic/versions/`) e pelo DDL em `db/`.

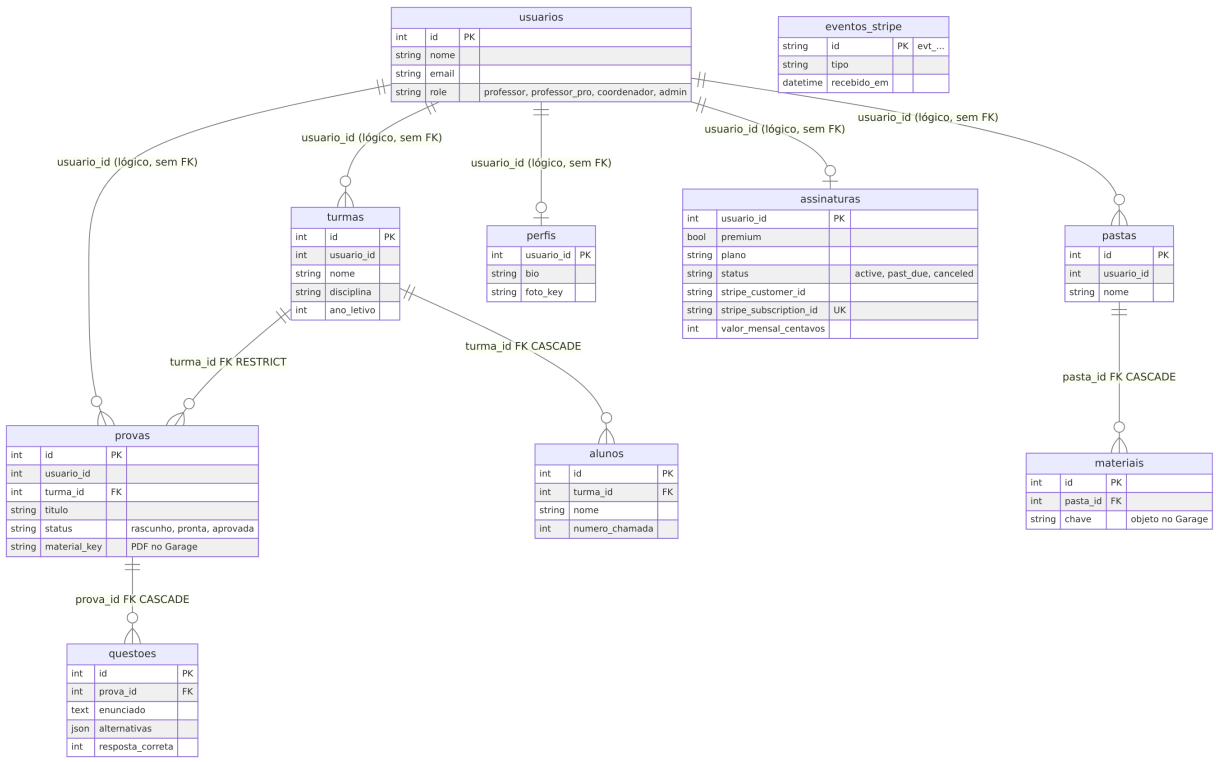


Figura 4: Modelo de dados: tabelas do domínio, chaves estrangeiras e a ligação usuário ⟶ assinatura do Stripe

3.6 Segurança

Ação	Free (`professor`)	Pro (`professor_pro`)	Escola (`coordenador`)
criar turmas, provas, PDFs	até 1 turma, 2 provas/mês, 3 PDFs	ilimitado	ilimitado
versão B embaralhada, gerar questões com IA	não (403)	sim	sim
aprovar prova, relatório da coordenação	não (403)	não (403)	sim
ver turma ou prova de outro usuário	não (404)	não (404)	não (404)
painel admin, trocar papéis, ler o log	não (403)	não (403)	não (403) — só `admin`

Os papéis formam uma escada: `professor` (Free) < `professor_pro` < `coordenador` (Escola) < `admin`, e cada um inclui o anterior. O papel vem só do JWT e quem barra é a API, com uma dependência que exige o papel mínimo; as telas apenas escondem botões. Toda ação acima do papel responde `403` também quando a API é chamada direto, e registra um evento `acesso_negado:*`. O admin tem ainda o painel com usuários, troca de papéis, eventos de auditoria e a visão geral do negócio.

O isolamento por usuário fica nas consultas: toda leitura de turma, prova, pasta ou PDF filtra pelo `usuario_id` do token, por uma função única (`get_owned_or_404`). O recurso de outra pessoa responde `404`, e não `403`, para não revelar que o ID existe. Nem o coordenador aprova prova de outro professor. Os arquivos no Garage só abrem por link pré-assinado, que expira e só é gerado para o dono.

Os segredos (`JWT_SECRET`, senha do banco, chaves do Stripe, do Garage, do SMTP e da IA) ficam só nas variáveis da stack no Portainer. O repositório tem apenas o `.env.example`, com os nomes e sem valores, e o `.env` está no `.gitignore`. O CI usa um `.env` descartável com valores aleatórios a cada execução.

3.7 Observabilidade, CI/CD e documentação da API

Fiz as três extras no 1º bimestre e elas continuam no produto:

- **Documentação da API:** o Swagger fica em `/docs`, com os erros de cada rota (401, 403, 404, 409, 422, 503) documentados com exemplo.
- **CI/CD:** o GitHub Actions roda os testes da api (199 hoje) e do log-service, faz o build das quatro imagens, sobe a stack inteira num smoke test que repete o fluxo de turma, prova, PDF, 404 e 403, publica as imagens no GHCR com a tag do commit e faz um commit automático (`deploy: sha-…`) apontando o compose de produção para essa tag. O Portainer puxa por GitOps; não há deploy manual.
- **Observabilidade:** todos os containers têm health check, e `/health` da api mostra o estado do MySQL, do auth-service, do log-service e do Garage. Cada serviço expõe `/metrics` (Prometheus) numa porta interna, a 9100, fora do domínio público. O Prometheus e o painel do Grafana sobem só no ambiente local (`docker-compose.yml`); em produção ficam os health checks e as métricas internas.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no seu projeto	Quem gerencia
IaaS	a VM da disciplina, a rede e o disco onde a stack roda	professor / Fatec
PaaS	Docker + Portainer, MySQL remoto, proxy HTTPS do domínio, GitHub Actions e GHCR; Redis e Garage rodam como containers da minha stack	professor (VM, Portainer, MySQL, domínio); GitHub (CI e registro); eu (containers)
SaaS	o ProvaPronta para os professores; e os SaaS que eu uso: Stripe, Resend, Groq, GitHub	eu (o produto); os fornecedores (os outros)

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	09/10/2026 16:33	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	23/10/2026 20:03	entregue com atraso
9	Fork e plano de negócio	23/10/2026	23/10/2026 22:39	entregue
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	03/11/2026 16:15	entregue
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	10/11/2026 20:19	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	09/10/2026 16:33
Situação	entregue
Evidência	GitHub — commit b0b0ace (repositório antigo `api_filme_`) e issue #10 do repositório antigo + prints do checkout concluído e do usuário premium
Link	https://github.com/leonardoricci-tsi/api_filme_/commit/b0b0ace022e08357cc02922b308040bafe9d43ce

O que foi feito. O catálogo de filmes ganhou planos pagos de verdade, no modo de teste do Stripe. Criei dois planos, Nerd e Stalker, cada um ligado a um papel do RBAC, e uma Checkout Session hospedada pelo Stripe, de modo que o número do cartão nunca chega ao servidor. O webhook `checkout.session.completed`, com a assinatura `Stripe-Signature` validada, é o único caminho que ativa o plano: ele pede ao auth-service a troca do papel e grava a assinatura no banco. O usuário gratuito ficou limitado a 5 favoritos, e o perfil passou a mostrar o selo do plano.

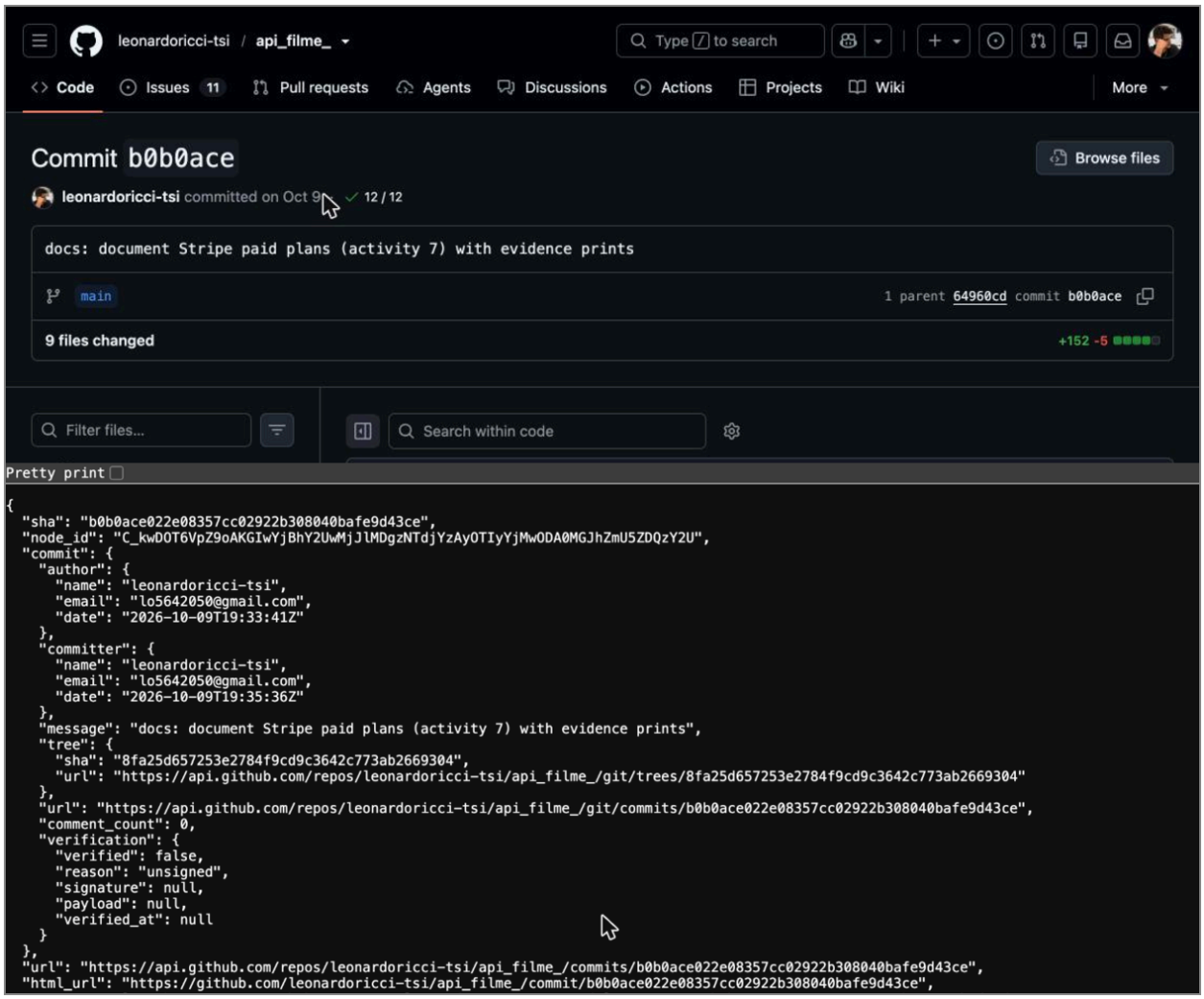


Figura 5: Atividade 7 — evidência da entrega: a página no GitHub e o mesmo registro na API do GitHub, com data e hora (UTC)

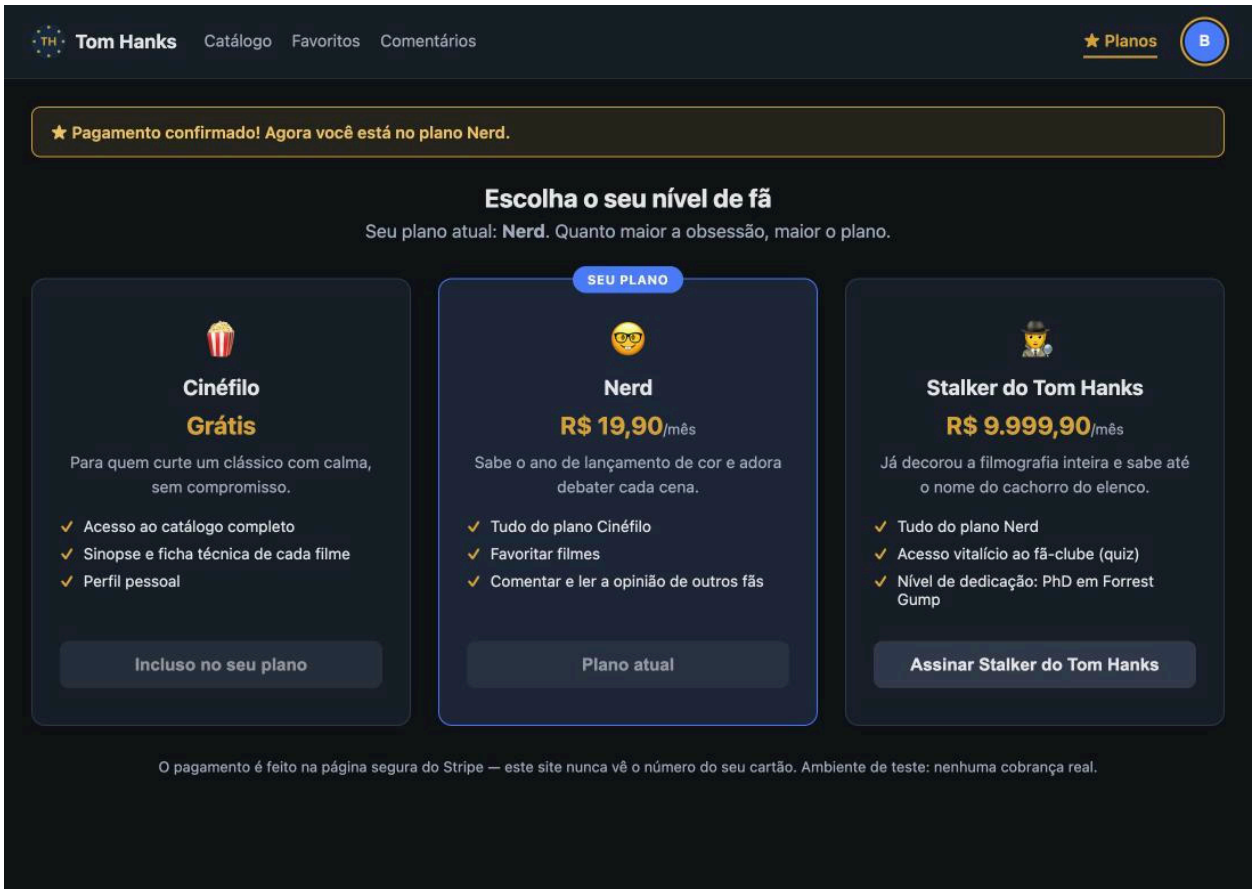


Figura 6: Atividade 7 — resultado: pagamento confirmado pelo webhook e usuário com plano pago

Mais prints do resultado:

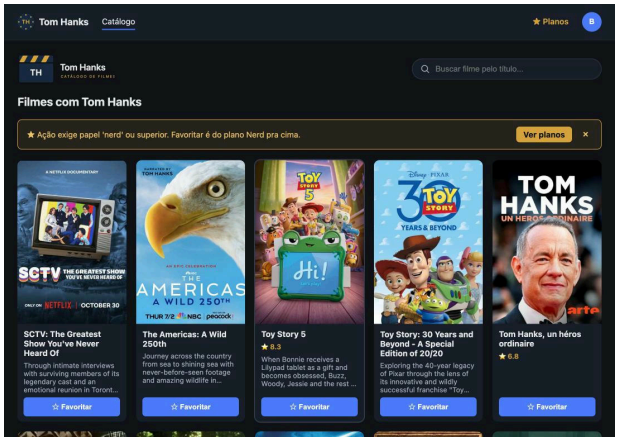


Figura 7: Atividade 7 — Usuário gratuito barrado no limite de favoritos (403)

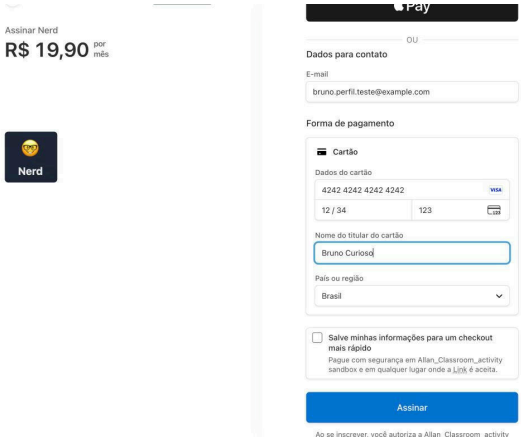


Figura 8: Atividade 7 — Checkout do Stripe em modo de teste

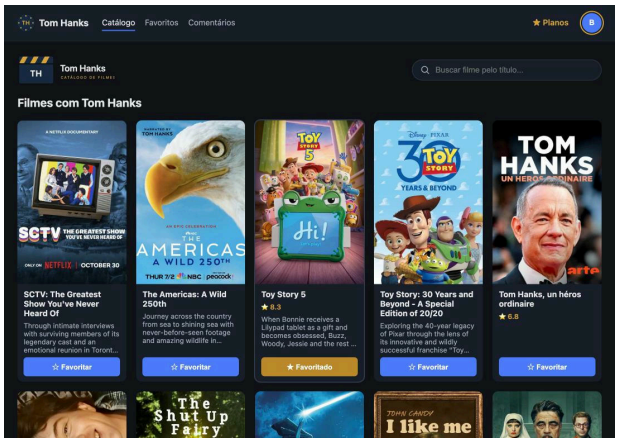


Figura 9: Atividade 7 — Com o plano pago, o mesmo favorito é aceito (201)

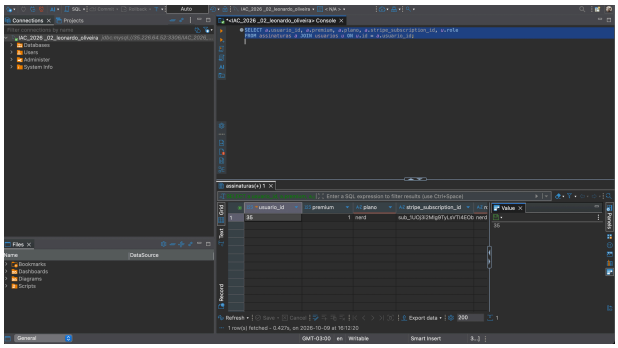


Figura 10: Atividade 7 — Assinatura gravada no banco e papel trocado pelo webhook

Dificuldades e como foram resolvidas. A parte mais delicada foi o webhook: a assinatura é calculada sobre o corpo bruto da requisição, então a rota tem que ler os bytes exatos que o Stripe mandou, sem o FastAPI interpretar o JSON antes. Também foi preciso aceitar que o navegador volta do checkout antes de o webhook chegar; a página de planos passou a consultar o status até o plano aparecer e a renovar o token para o papel novo valer.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	23/10/2026 20:03
Situação	entregue com atraso
Evidência	GitHub — commit f9c19da (repositório antigo) + DOCUMENTACAO.md linkado no README
Link	https://github.com/leonardoricci-tsi/api_filme_/commit/f9c19da6d60c294f987b7de491b66b6ab7fb9bba

O que foi feito. Documentei o sistema como se ele tivesse sido herdado de outra pessoa, sem escrever código novo. O `DOCUMENTACAO.md` traz a especificação, a arquitetura, as regras de negócio extraídas do código (cada uma com o arquivo de onde saiu), o modelo de dados e uma seção de achados da leitura, onde o código fazia algo diferente do que o README dizia. No README ficou a classificação de cada peça do projeto nas camadas SaaS, PaaS e IaaS, com o link para a documentação.

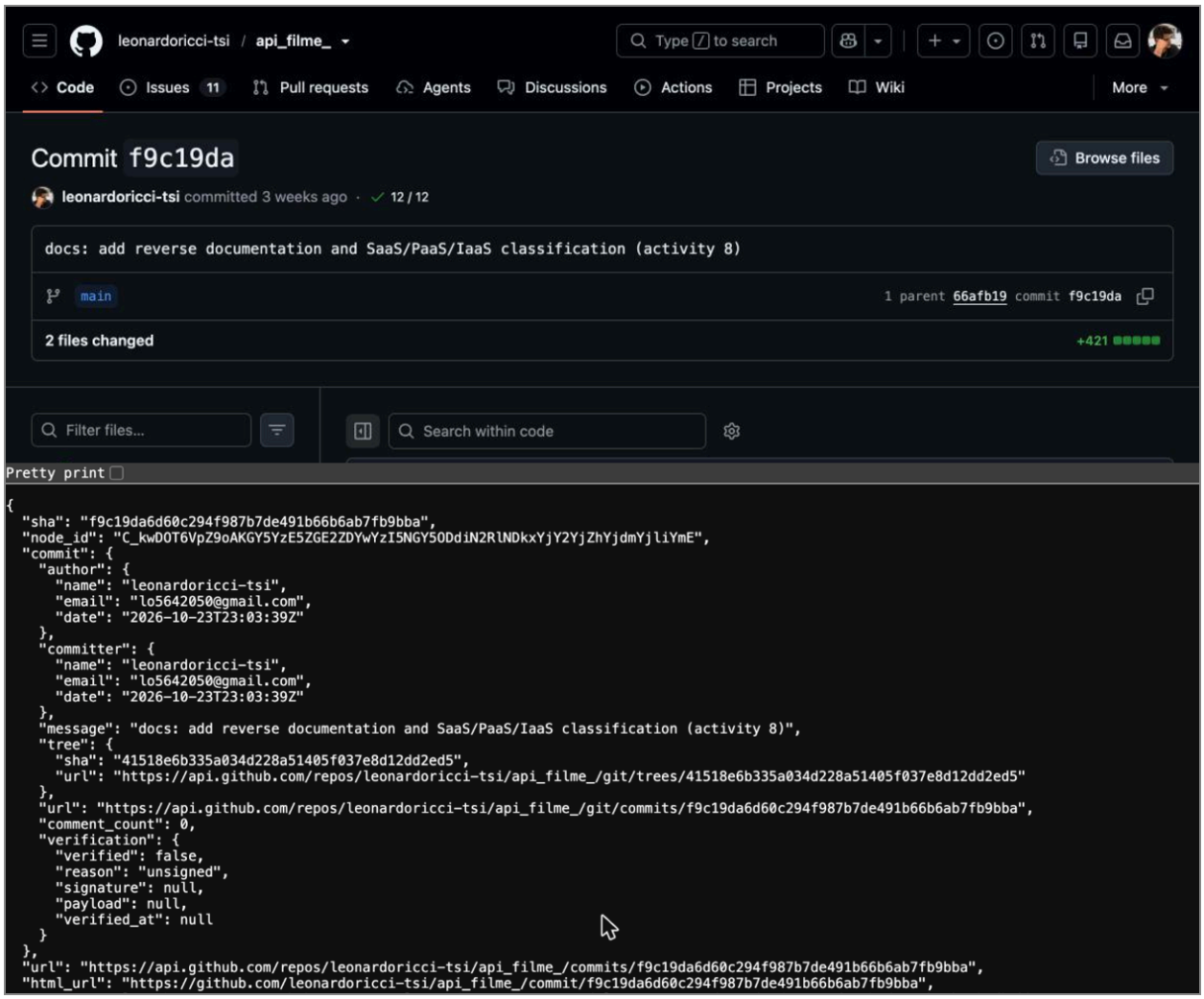


Figura 11: Atividade 8 — evidência da entrega: a página no GitHub e o mesmo registro na API do GitHub, com data e hora (UTC)

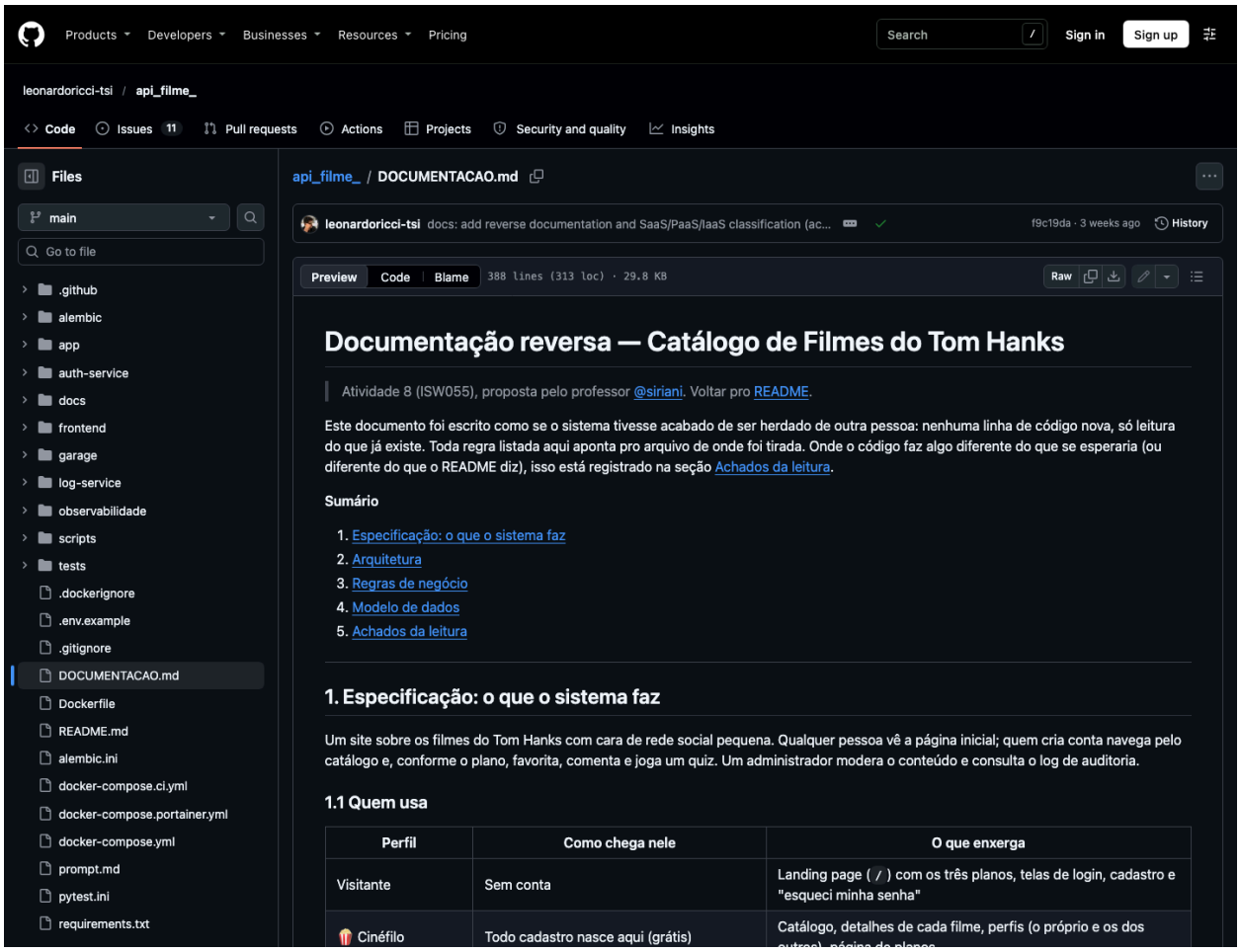


Figura 12: Atividade 8 — resultado: DOCUMENTACAO.md no repositório, linkada pelo README

Mais prints do resultado:

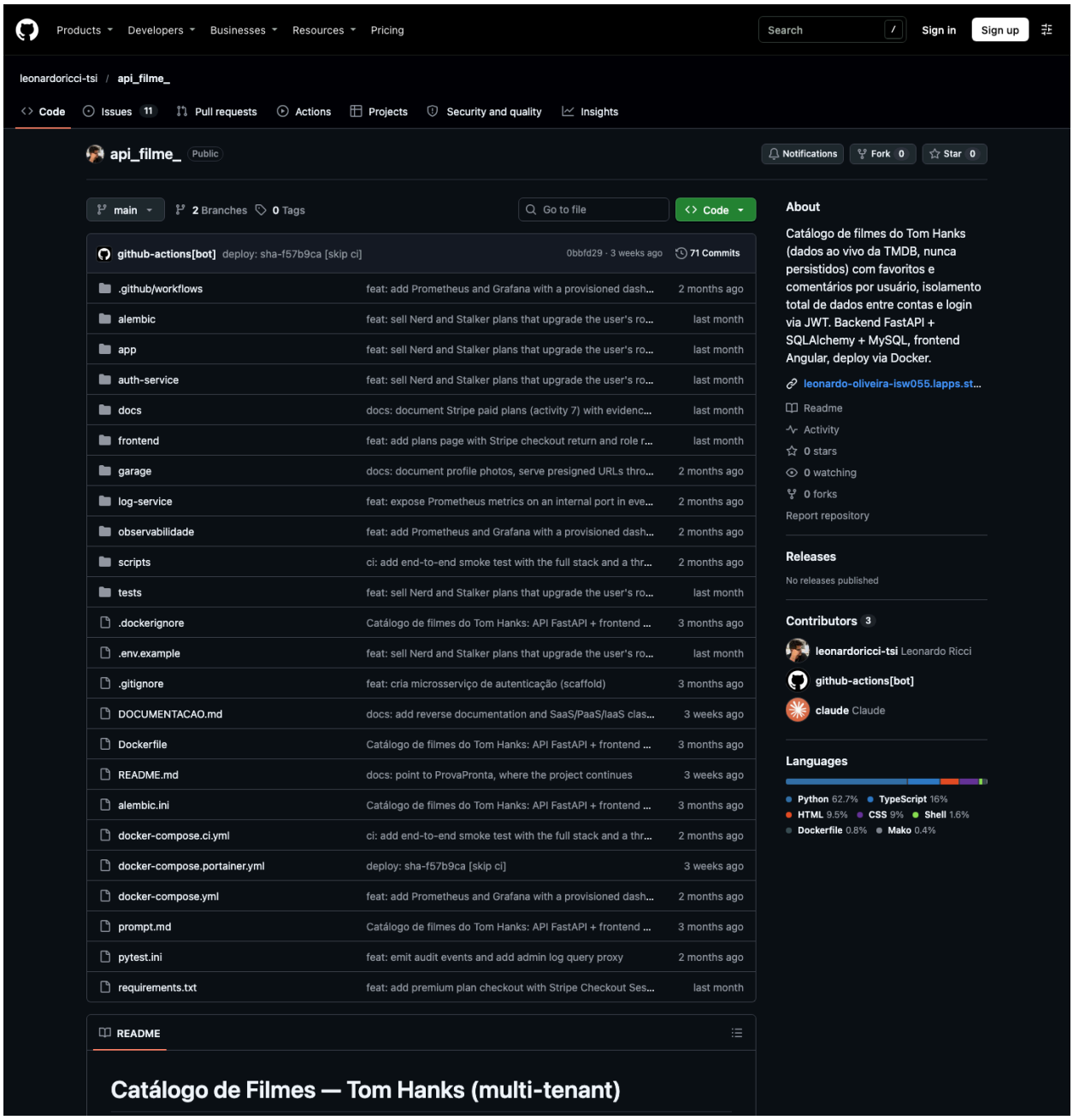


Figura 13: Atividade 8 — README do repositório antigo com a classificação nas camadas e o link para a DOCUMENTACAO.md

Dificuldades e como foram resolvidas. Atraso: a atividade foi entregue em 23/10, uma semana depois do prazo de 16/10. Atrasei a entrega porque achei que ela já tinha sido feita junto com o relatório da P1; depois que o professor avisou em sala, fiz a atividade, já com atraso. A maior dificuldade foi a própria leitura reversa: separar o que o código realmente faz do que a documentação antiga prometia, e classificar nas camadas de nuvem peças que dependem de quem gerencia cada uma (o mesmo container é PaaS para mim porque a VM e o Docker já existiam).

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.

Data planejada	23/10/2026
Data realizada	23/10/2026 22:39
Situação	entregue
Evidência	GitHub — issue #15 “Entrega — Fork e plano de negócio” (@siriani) + repositório novo + README do produto + histórico e issues
Link	https://github.com/leonardoricci-tsi/ProvaPronta/issues/15

O que foi feito. Escolhi o domínio do produto e criei o repositório novo, `ProvaPronta`, copiando o antigo com todo o histórico (`git clone --bare` e `git push --mirror`), para que os commits do 1º bimestre e das atividades 7 e 8 continuassem verificáveis. O README do produto ganhou a dor, a persona (a professora Marta, com oito turmas), os concorrentes, a proposta de valor, o modelo de receita com Free, Pro e Escola, o Lean Canvas e o mapa do que seria reaproveitado do catálogo. O backlog virou 14 issues, separadas pelas atividades seguintes, e o repositório antigo ganhou no topo um aviso apontando para o novo.

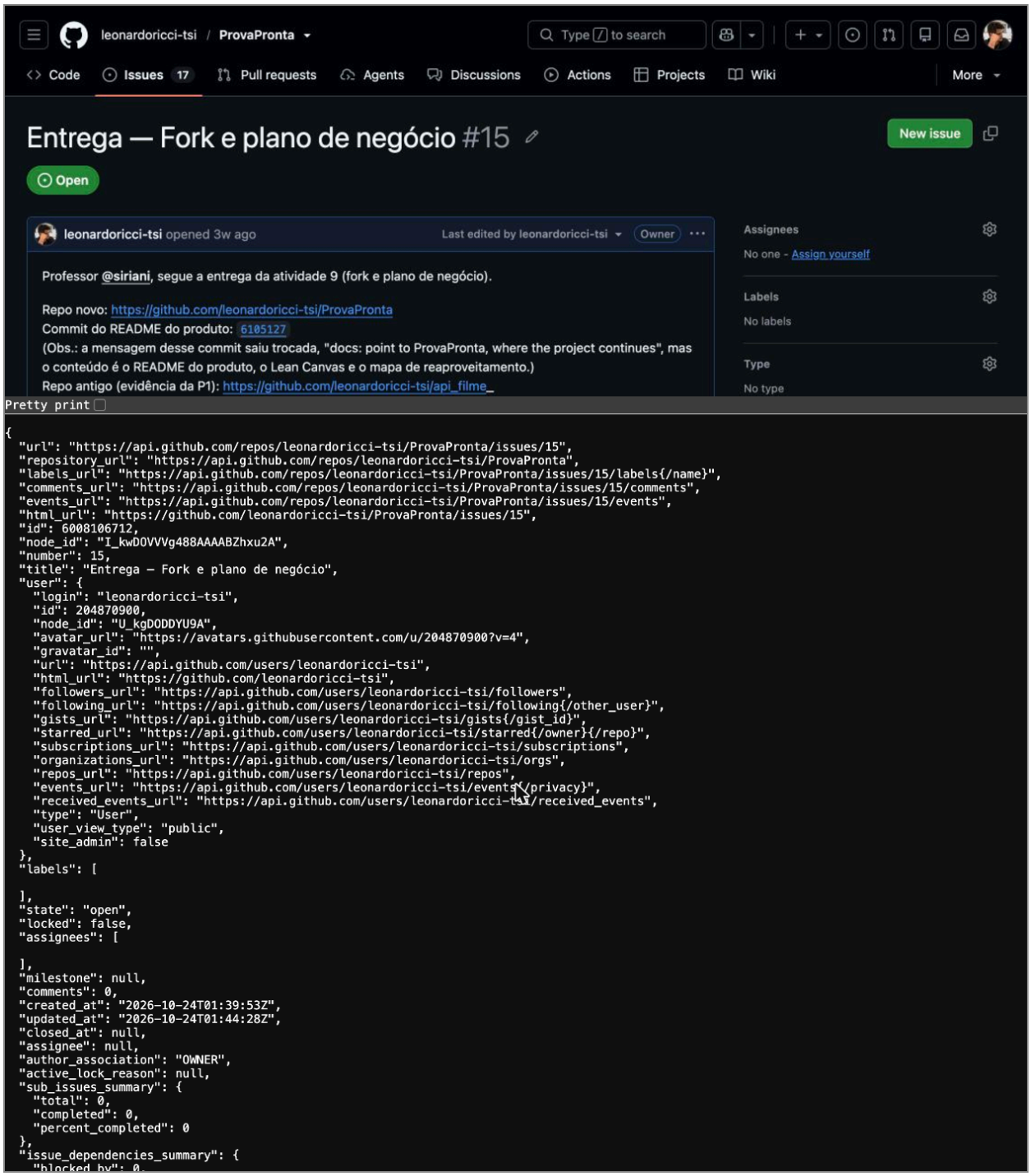


Figura 14: Atividade 9 — evidência da entrega: a página no GitHub e o mesmo registro na API do GitHub, com data e hora (UTC)

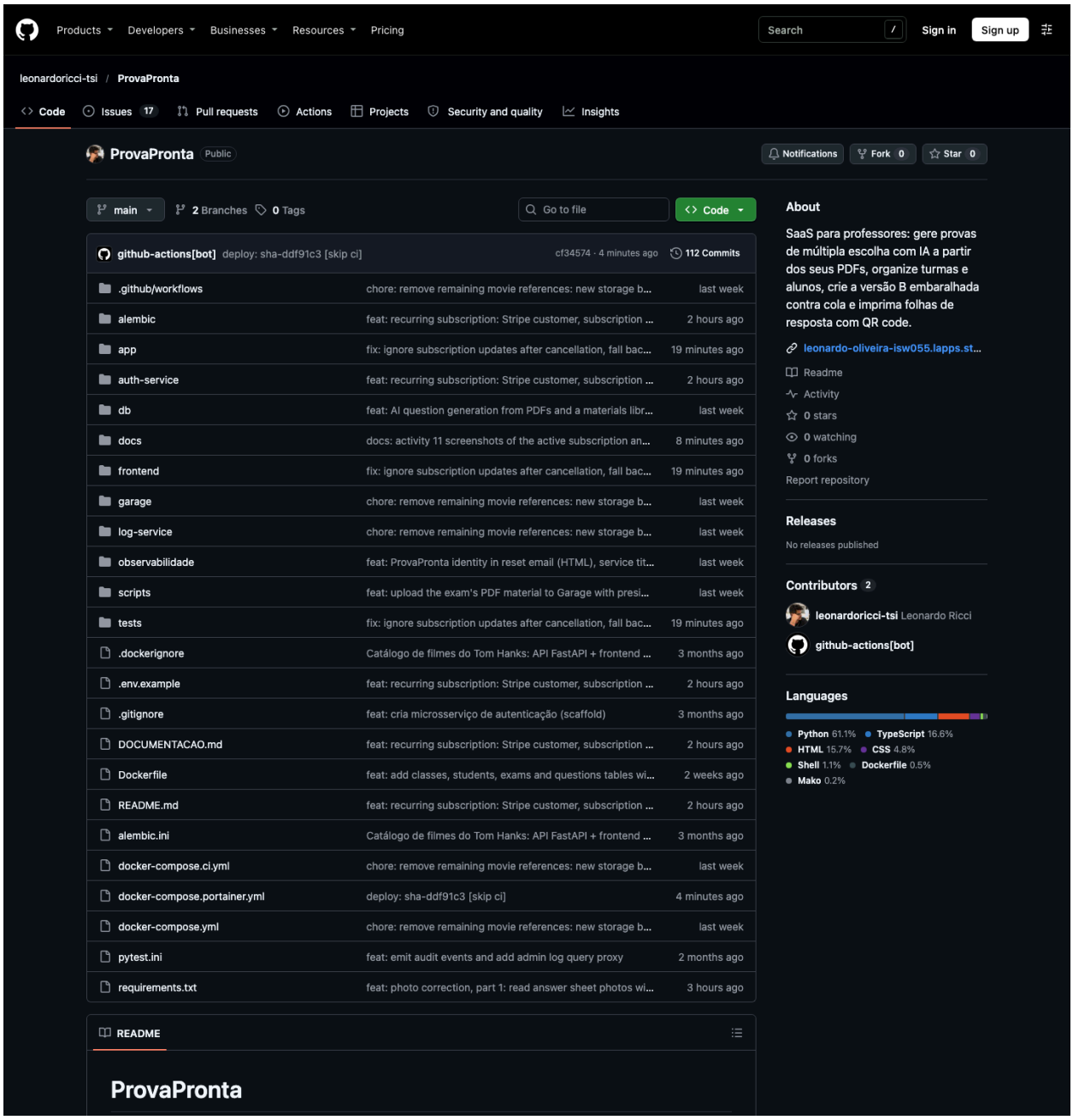


Figura 15: Atividade 9 — resultado: README do produto no repositório novo

Mais prints do resultado:

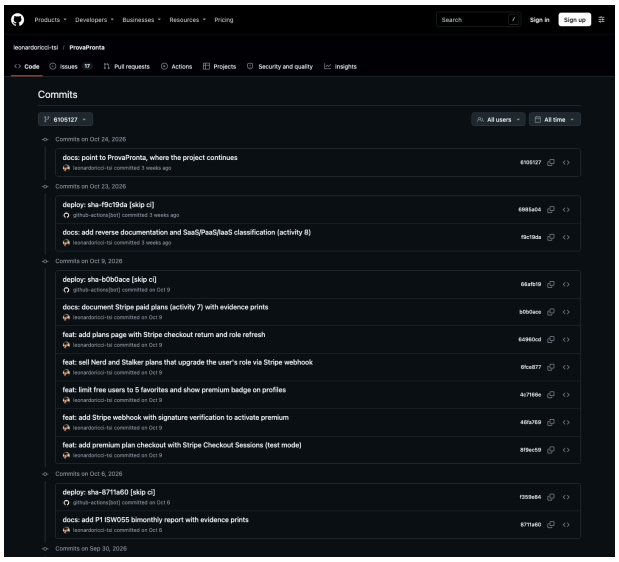


Figura 16: Atividade 9 — Histórico copiado: os commits das atividades 7 e 8 no repositório novo

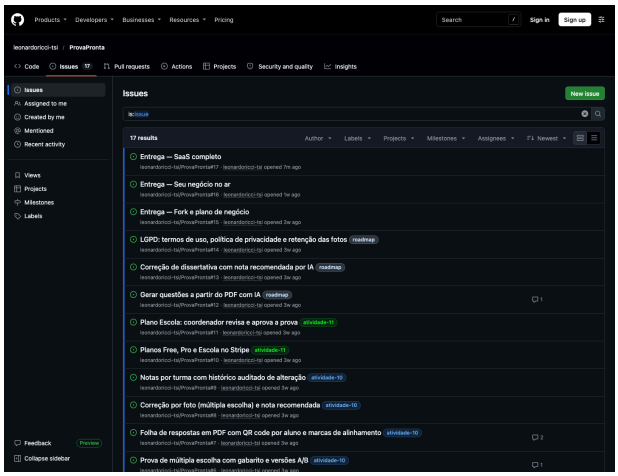


Figura 17: Atividade 9 — Backlog do produto em issues, com as entregas

Dificuldades e como foram resolvidas. A dificuldade foi escolher um produto que reaproveitasse a infraestrutura que eu já tinha (login separado, papéis, auditoria, upload e Stripe) e que tivesse uma dor real e um diferencial claro. Também foi preciso fazer a cópia do repositório de um jeito que preservasse o histórico, em vez de começar do zero.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	03/11/2026 16:15
Situação	entregue
Evidência	GitHub — issue #16 “Entrega — Seu negócio no ar” (@siriani) + commit + URL no ar + prints da landing, do 403 e dos eventos
Link	https://github.com/leonardoricci-tsi/ProvaPronta/issues/16

O que foi feito. Troquei os filmes pelo domínio do ProvaPronta no mesmo endereço. A stack do Portainer passou a puxar do repositório novo, com as imagens renomeadas para `provapronta-*`. Criei as tabelas do domínio (turmas, alunos, provas e questões) com chaves estrangeiras, a API e as telas de cada uma, os papéis do negócio (Free, Pro, Escola e admin) com ações exclusivas para cada um, o isolamento por usuário com resposta 404, os eventos de negócio no log, o upload do PDF da apostila no Garage e a landing page com os planos. Depois da entrega o produto continuou crescendo: biblioteca de materiais em pastas, geração de questões com IA, folhas de resposta com QR code e e-mails pelo Resend com domínio próprio.

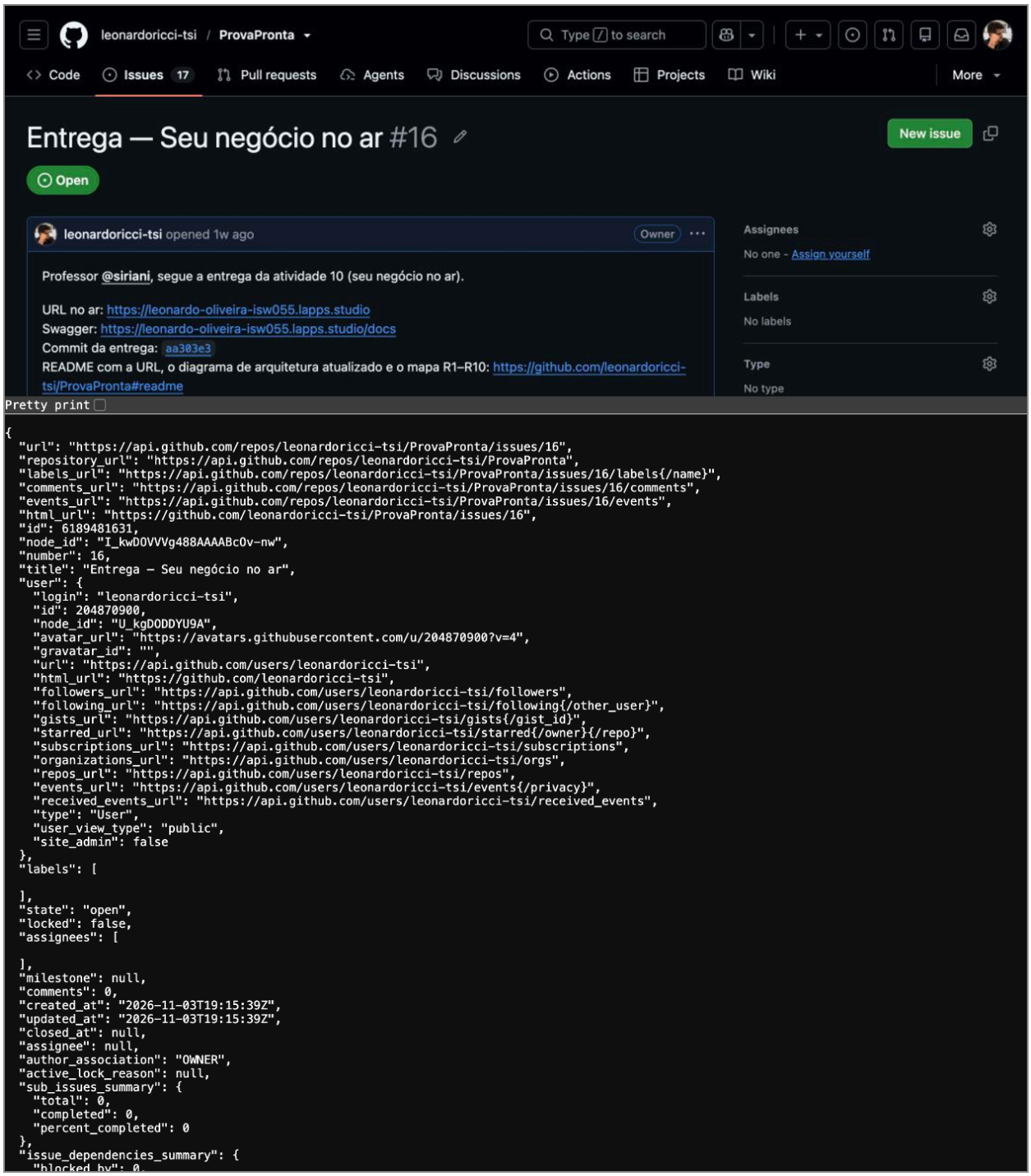


Figura 18: Atividade 10 — evidência da entrega: a página no GitHub e o mesmo registro na API do GitHub, com data e hora (UTC)

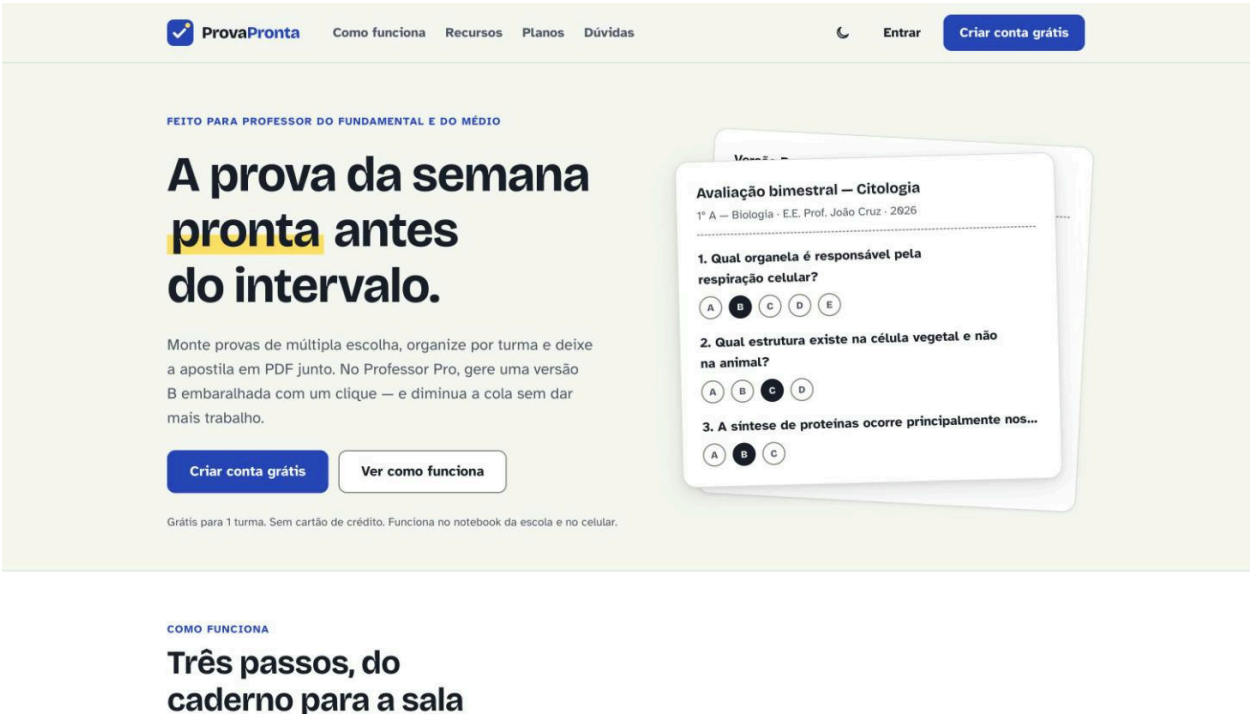


Figura 19: Atividade 10 — resultado: landing do ProvaPronta no ar

Mais prints do resultado:

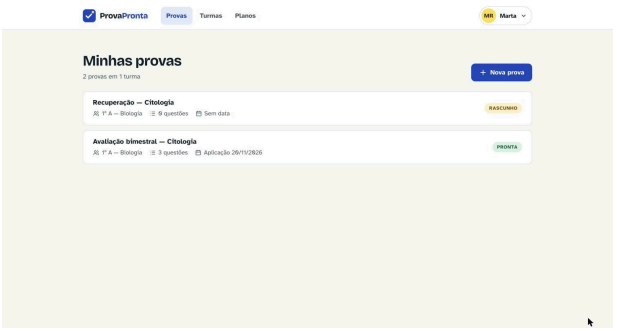


Figura 20: Atividade 10 — Listagem de provas da professora (plano Pro)

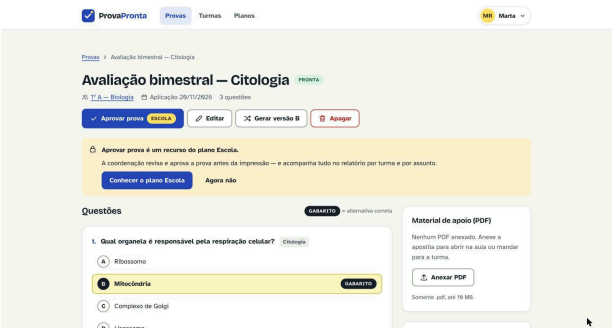


Figura 21: Atividade 10 — 403: o Pro tenta aprovar uma prova, ação do plano Escola

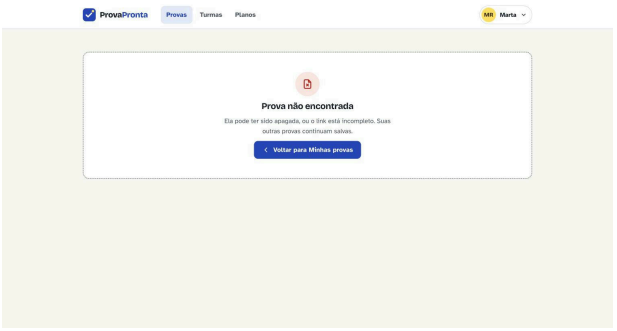


Figura 22: Atividade 10 — 404: prova de outro usuário

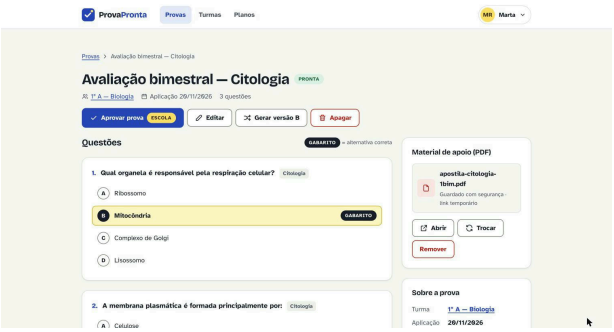


Figura 23: Atividade 10 — PDF da apostila anexado à prova

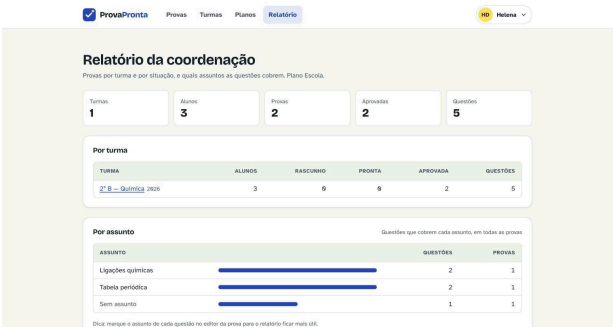


Figura 24: Atividade 10 — Relatório da coordenação (plano Escola)

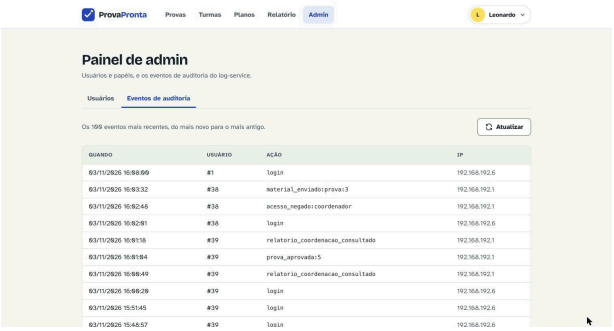


Figura 25: Atividade 10 — Eventos de negócio no log-service

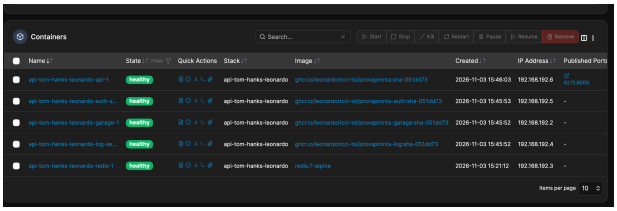


Figura 26: Atividade 10 — Os containers da stack no Portainer, saudáveis

Dificuldades e como foram resolvidas. Para mim foi a atividade mais difícil, porque foi preciso mudar toda a infraestrutura para o projeto novo sem perder o que já estava no ar. O Portainer CE não deixa trocar o repositório de uma stack Git, então ela teve que ser removida e recriada com o mesmo nome, as mesmas variáveis e os mesmos volumes. Também foi preciso criar um bucket novo no Garage, refazer as integrações (Stripe, e-mail, chaves da IA) nas variáveis da stack, aplicar as migrations do banco pelo console do container e descobrir que o Cloudflare na frente do domínio trocava as respostas 502 da api pela página de erro dele, o que escondia a mensagem real.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	10/11/2026 20:19
Situação	entregue
Evidência	GitHub — issue #17 “Entrega — SaaS completo” (@siriani) + commits 3fa8ebf e 90faeb5 + prints da assinatura com o cartão 4242, do webhook e do limite
Link	https://github.com/leonardoricci-tsi/ProvaPronta/issues/17

O que foi feito. O pagamento virou assinatura recorrente. O checkout passou a criar e guardar o customer do Stripe do usuário, os webhooks passaram a tratar `customer.subscription.updated` e `invoice.payment_failed` além dos dois que já existiam, e cada evento processado é registrado para não ter efeito duas vezes. Entraram o portal do cliente (“Gerenciar assinatura”), com troca de plano e cancelamento imediato, o aviso de pagamento recusado por e-mail e na tela, e a visão geral do painel admin, com usuários por plano, assinaturas ativas, inadimplentes, MRR de teste e últimos eventos. Os limites do Free continuaram barrados na API.

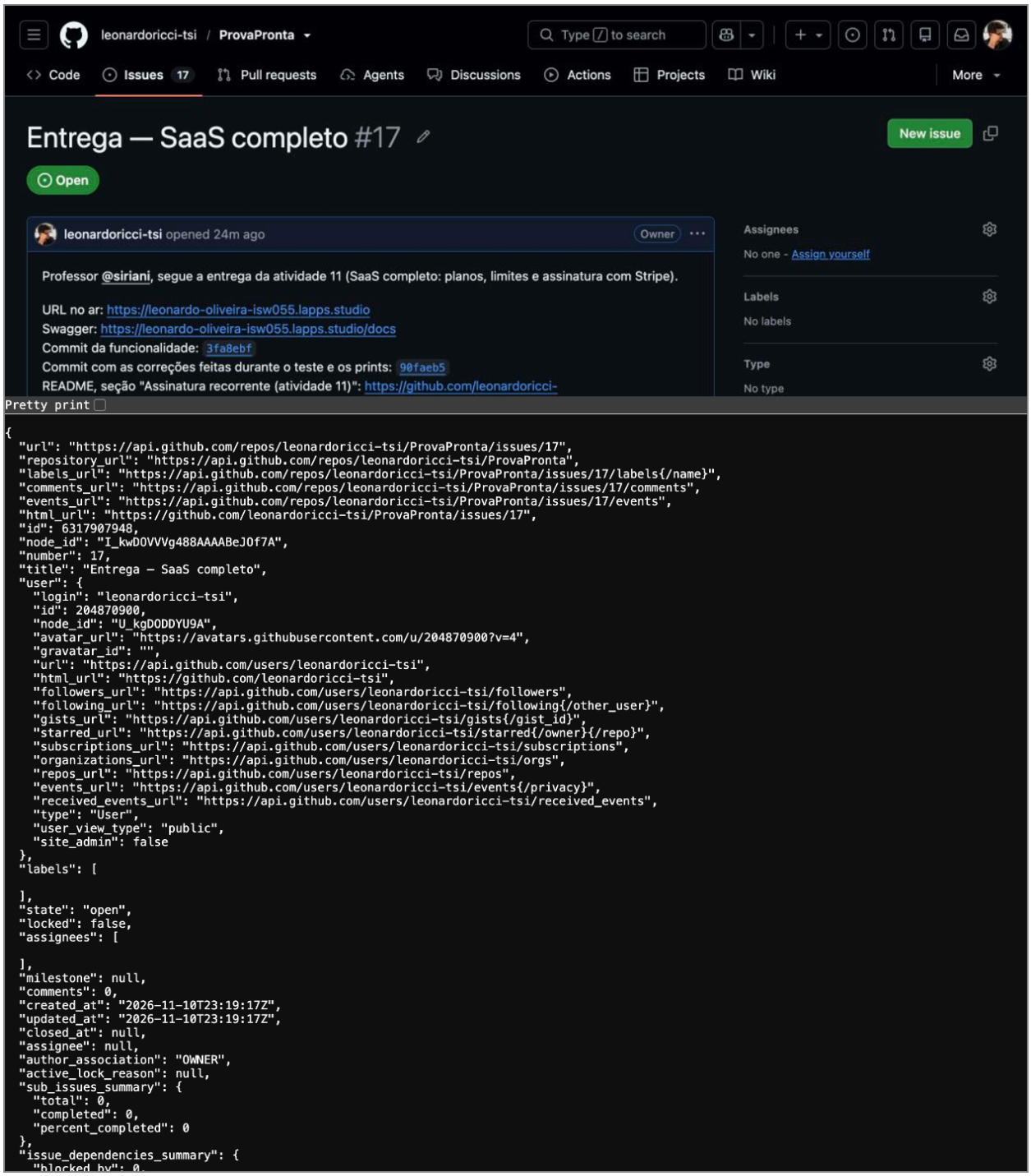


Figura 27: Atividade 11 — evidência da entrega: a página no GitHub e o mesmo registro na API do GitHub, com data e hora (UTC)

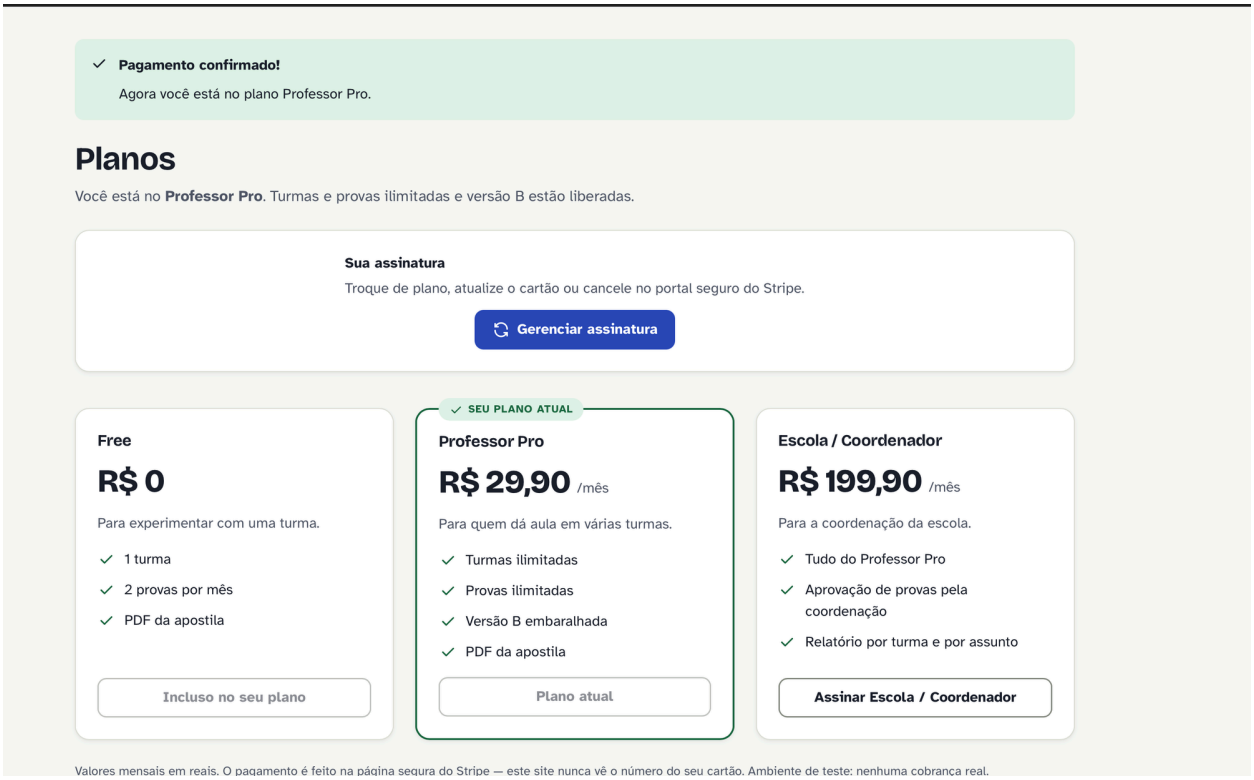


Figura 28: Atividade 11 — resultado: assinatura do Pro confirmada pelo webhook

Mais prints do resultado:

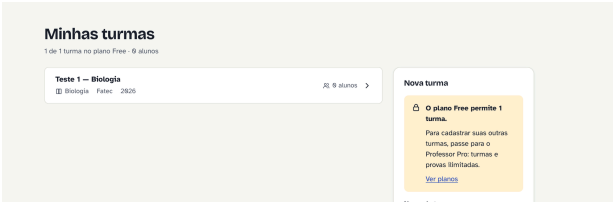


Figura 29: Atividade 11 — Limite do Free bloqueando a segunda turma

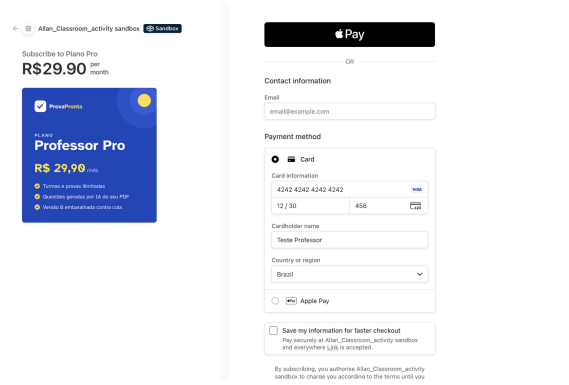


Figura 30: Atividade 11 — Assinatura do Pro com o cartão de teste 4242

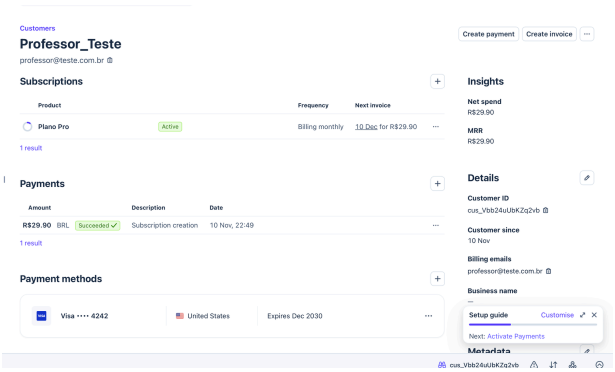


Figura 31: Atividade 11 — Assinatura ativa no painel do Stripe (modo de teste)

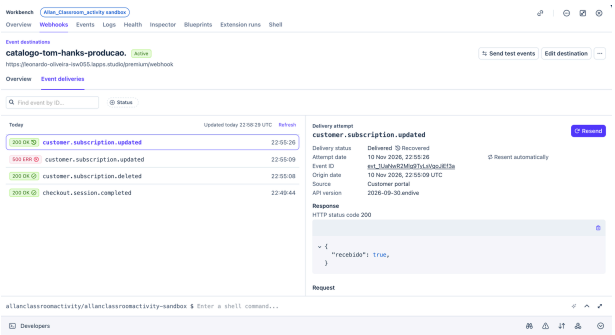


Figura 32: Atividade 11 — Webhooks entregues no endpoint /premium/webhook

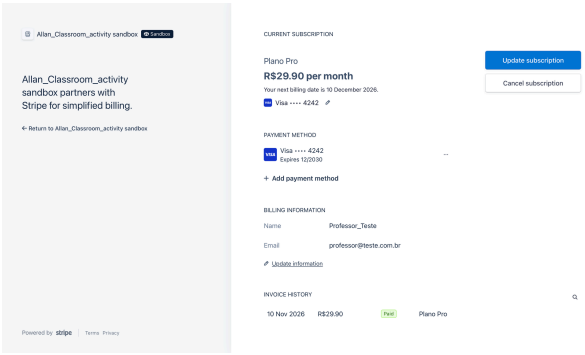


Figura 33: Atividade 11 — Portal do cliente do Stripe

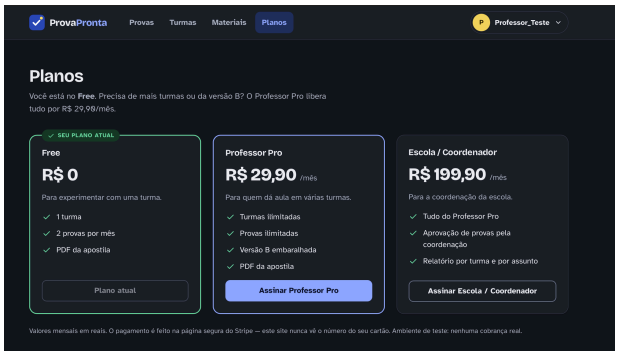


Figura 34: Atividade 11 — Depois do cancelamento, a conta volta ao Free

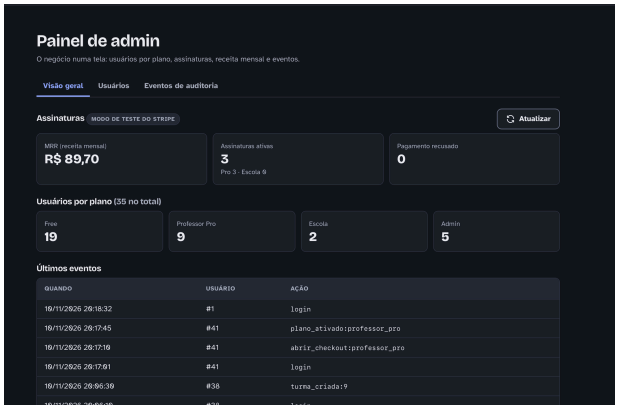


Figura 35: Atividade 11 — Painel admin: usuários por plano, assinaturas e MRR de teste

Dificuldades e como foram resolvidas. O deploy travou porque o disco do servidor da disciplina encheu (`no space left on device` ao clonar o repositório); com a orientação do professor, removi as imagens antigas da minha stack e o deploy voltou. No teste real, um evento `customer.subscription.updated` voltou com erro 500 na primeira tentativa e o Stripe reenviou sozinho com sucesso; ao investigar, vi que os eventos do portal podem chegar fora de ordem e corrigi para que uma atualização atrasada não reative uma assinatura cancelada. Também foi preciso configurar o portal do cliente no Stripe, que sem os produtos cadastrados não oferece a troca de plano.

6 Considerações finais

O bimestre mudou o jeito como eu olho para o projeto. No primeiro, cada atividade era uma peça isolada; no segundo, as mesmas peças tiveram que funcionar juntas para sustentar um produto que cobra, limita e audita. Aprendi que a parte mais trabalhosa de um SaaS não é a tela, e sim o que fica em volta dela: mudar toda a infraestrutura para o projeto novo sem derrubar o que estava no ar, integrar Stripe, e-mail e IA pelas variáveis do Portainer, aplicar migrations em produção e lidar com problemas que só aparecem no servidor real, como o Cloudflare escondendo erros, o disco cheio e eventos do Stripe chegando fora de ordem. Também ficou claro o valor do que foi construído no 1º bimestre: CI/CD, testes e logs foram o que me deixou mudar tanta coisa com segurança.

Para o ProvaPronta ser um produto de verdade ainda falta o principal diferencial: a correção por foto completa, com a tela de revisão e as notas da turma (a leitura da folha já existe na API). Também faltam a importação de alunos por planilha, o banco de questões reaproveitável, o cumprimento da LGPD (termos de uso, política de privacidade e retenção das fotos), sair do modo de teste do Stripe e rodar numa infraestrutura própria, com limites de memória, backup e monitoramento em produção.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

gov.br
Documento assinado digitalmente
LEONARDO RICCI SANTOS OLIVEIRA
Data: 10/11/2026 21:10:53-0300
Verifique em https://validar.iti.gov.br

Leonardo Ricci Santos Oliveira

Pompeia, 10/11/2026